# TP Math Informatique

Tonin PACHER

April 21, 2026

## Contents

**1 Matrice 2D** **2**
1.1 Codage de l'Entropie H(X) . . . . . 2
1.2 Codage de l'Entropie jointe H(X,Y) . . . . . 2
1.3 Codage de l'Entropie conditionnelle H(X|Y) . . . . . 3
1.4 Codage de l'information mutuelle I(X;Y) . . . . . 3

**2 Matrice 3D** **4**
2.1 Codage de l'entropie de la dernière variable 3D . . . . . 4
2.2 Codage de l'entropie conditionnelle jointe de H(A,B|C) . . . . 5
2.3 Codage de l'information mutuelle I(A,B;C) . . . . . 5

**3 Hypercube** **6**
3.1 Codage de l'information mutuelle dirigée . . . . . 6
3.2 Codage de l'information dirigée sur la matrice 3D . . . . . 6

**4 Étude de données réelles sur des cachalots** **7**
4.1 Récupération des données . . . . . 7
4.2 Mise en place de la synchronisation . . . . . 8
4.2.1 Visualisation du Tenseur $W$ . . . . . 12
4.3 Identification de l'alpha par l'Information Dirigée . . . . . 13
4.3.1 Analyse des résultats numériques et démonstration des limites . . . . . 14
4.4 L'Égalité de Massey : Causalité vs Corrélation . . . . . 16
4.5 Ajout du glissement pour améliorer les résultats . . . . . 20
4.5.1 Principe du glissement . . . . . 20
4.5.2 Implémentation . . . . . 20
4.5.3 Calcul des scores locaux . . . . . 21
4.5.4 Pourquoi utiliser des fenêtres glissantes ? . . . . . 21
4.5.5 Interprétation . . . . . 22
4.6 Conclusion du rapport et corrélation avec l'aspect biologique . 22
4.6.1 Analyse biologique : Leadership tactique et mémoire collective . . . . . 23

4.6.2 Impact du temps d'émission sur l'identification de l'alpha 23
4.6.3 Comparaison 6 animaux vs 3 animaux . . . . . 24
4.7 Perspectives futurs . . . . . 24

# 1 Matrice 2D

Toutes les matrices aléatoires seront de taille $(8 \times 16)$.

On peut donc déjà définir les bornes maximales pour les différentes entropies :

$$\max H(X) = \log_2(8) = 3$$

$$\max H(Y) = \log_2(16) = 4$$

$$\max H(X,Y) = \log_2(128) = 7$$

$$\max I(X;Y) = \min\big(H(X),\ H(Y)\big)$$

## 1.1 Codage de l'Entropie H(X)

Première étape : nous allons coder l'entropie classique se basant sur une seule variable aléatoire, ici $H(X)$.

Rappelons la formule :

$$H(X) = -\sum_i p(x_i) \times \log_2\big(p(x_i)\big)$$

**Le code Python est donc :**

```python
def Entropie(Z):
    """ entropie de H(X)"""
    Z = Z + eps
    Z = Z / Z.sum()
    Px = np.sum(Z, axis=1)
    return -np.sum(Px * np.log2(Px))
```

Après exécution sur une matrice 2D, **cela nous donne : 2.9875584206597683**

La borne maximale notée plus haut est 3, c'est donc cohérent étant donné qu'on est sur un tirage aléatoire : l'incertitude est très élevée. Après test sur un Dirac, on obtient bien 0.

## 1.2 Codage de l'Entropie jointe H(X,Y)

Deuxième étape : nous allons coder l'entropie jointe. On rappelle la formule :

$$H(X,Y) = -\sum_{i,j} p(x_i, y_j) \times \log_2\big(p(x_i, y_j)\big)$$

**Le code Python est donc :**

```python
def Entropie_jointe(Z):
    """ entropie jointe entre H(X) et H(Y) """
    Z = Z + eps
    P_Z = Z / np.sum(np.ndarray.flatten(Z))
    P_flat = P_Z.flatten()
    return -1 * np.sum((P_flat) * np.log2(P_flat))
```

Après exécution sur une matrice 2D : **cela nous donne : 6.746527033542511**

C'est cohérent car la borne maximale notée plus haut est 7, et comme on fait un tirage aléatoire on se rapproche de la borne maximale car l'incertitude est élevée.

### 1.3 Codage de l'Entropie conditionnelle H(X|Y)

Pour coder l'entropie conditionnelle, nous allons utiliser les formules rédigées au-dessus. On rappelle la formule de l'entropie conditionnelle :

$$H(X \mid Y) = H(X, Y) - H(Y)$$

**Le code Python est donc :**

```python
def Entropie_conditionnel(Z):
    """ calcul de l'entropie conditionnel H(X | Y)"""
    J = Entropie_jointe(Z)
    E = Entropie_Y(Z)
    return J - E
```

Il faut prendre en compte que la fonction `Entropie_Y` existe ; il faut juste changer la valeur de `axis` dans l'entropie de X.

Après exécution sur la même matrice 2D :

**nous donne : 3.7670756143714543**

### 1.4 Codage de l'information mutuelle I(X;Y)

Pour coder l'information mutuelle de $X$ et $Y$, nous allons utiliser les formules rédigées à l'avance pour des questions de facilité. On rappelle que la formule est :

$$I(X;Y) = H(X) + H(Y) - H(X, Y)$$

**Le code Python est donc :**

```python
def Information_Mutuelle(Z):
    """ calcul de l'information mutuelle I(X;Y) """
    J = Entropie_jointe(Z)
    Ex = Entropie(Z)
    Ey = Entropie_Y(Z)
    return Ex + Ey - J
```

Après exécution sur la même matrice 2D :

**nous donne : 0.21092405738769138**

On sait que la borne maximale de l'information mutuelle est la plus petite valeur entre $H(X)$ et $H(Y)$. Après calcul, on remarque que la valeur est très basse ; on peut donc en conclure que les variables $X$ et $Y$ n'ont pas beaucoup de lien.

# 2 Matrice 3D

Nous serons tout au long sur une matrice 3D de taille $(8 \times 16 \times 7)$.

On peut donc définir les bornes maximales :

$$\max H(A) = \log_2(8) = 3$$

$$\max H(B) = \log_2(16) = 4$$

$$\max H(C) = \log_2(7) \approx 2{,}807$$

$$\max H(A, B) = \log_2(128) = 7$$

$$\max H(A, B, C) = \log_2(896) \approx 9{,}8$$

$$\max H(A, B \mid C) = \log_2(128) = 7$$

## 2.1 Codage de l'entropie de la dernière variable 3D

Pour calculer l'entropie de la 3D, il faut faire la somme de l'axe 1 puis, avec ces résultats, re-sommer sur l'axe 0 afin d'obtenir le total de la 3D. Attention à bien renormaliser après calcul.

$$H(C) = -\sum_{c=0}^{C-1} \left( \frac{\sum_{a,b}\big(Z[a,b,c] + \varepsilon\big)}{\sum_{a,b,c}\big(Z[a,b,c] + \varepsilon\big)} \right) \log_2 \left( \frac{\sum_{a,b}\big(Z[a,b,c] + \varepsilon\big)}{\sum_{a,b,c}\big(Z[a,b,c] + \varepsilon\big)} \right)$$

La fonction est un peu différente de l'entropie 2D car on compresse les axes que l'on ne veut pas. Étant donné qu'ici on est en 3D, il y a deux axes à supprimer : l'axe 0 et l'axe 1.

**Le code Python est donc :**

```python
def Entropie(Z): #calcul entropie peut importe la dimension du cube
    Z = np.array(Z, dtype=float) + eps
    Z = Z / Z.sum()
    Px = np.sum(Z, axis=1)
    return -np.sum(Px * np.log2(Px))
```

Les matrices 3D étant trop volumineuses pour être reproduites ici, le résultat obtenu sur une matrice générée aléatoirement est :

**2.806586742493664**

On remarque qu'on est très proche de la borne maximale ; on a donc une incertitude quasi-maximale.

## 2.2 Codage de l'entropie conditionnelle jointe de H(A,B|C)

Pour calculer cela, on va utiliser des fonctions écrites auparavant. La formule est :

$$H(A, B \mid C) = H(A, B, C) - H(C)$$

**Le code Python est donc :**

```
def Entropie_jointe(Z): #calcul de l'entropie (X,Y)
    Z = np.array(Z, dtype=float) + eps
    P_Z = Z / np.sum(Z)
    return -np.sum(P_Z.flatten() * np.log2(P_Z.flatten()))
```

On applique simplement la formule avec les fonctions codées auparavant et on obtient :

**6.7098072396548885**

On a donc un résultat très proche du maximum ; on peut donc en déduire que $A$ et $B$ dépendent très peu de $C$.

## 2.3 Codage de l'information mutuelle I(A,B;C)

Pour cela, on aura besoin de la fonction $H(A, B)$ :

```
def Entropie_jointe2D(Z1, axis):
    """ Entropie jointe de deux variables
    en marginalisant la troisieme """
    Z1 = Z1 + eps
    Z1 = Z1 / Z1.sum()
    P_2D = np.sum(Z1, axis=axis)
    P_2D = P_2D / np.sum(P_2D)
    P_flat = P_2D.flatten()
    return -np.sum(P_flat * np.log2(P_flat))
```

Ensuite on applique la formule :

$$I(A, B; C) = H(A, B) + H(C) - H(A, B, C)$$

**Le code Python est donc :**

```
def Information_Mutuelle(Z): #calcul info mutuelle avec formule
    return Entropie(Z) + Entropie_Y(Z) - Entropie_jointe(Z)
```

On obtient : **0.24095957591253025**

# 3 Hypercube

## 3.1 Codage de l'information mutuelle dirigée

La fonction `info_dirigee` calcule la somme des informations mutuelles conditionnelles de la forme :

$$ID(X^N \to Y^N) = \sum_{n=1}^{N} I(X^n; Y_n \mid Y^{n-1}) \tag{1}$$

où $I(A; B \mid C)$ représente l'information mutuelle conditionnelle entre $A$ et $B$ sachant $C$, et $N$ est le nombre de pas temporels (ici $N = 7$).

**Après développement de la formule nous obtenons :**

1. **Définition par l'Information Mutuelle Conditionnelle :**

$$ID(X \to Y) = \sum_{i=1}^{n} I(X^{i-1}; Y_i \mid Y^{i-1}) \tag{2}$$

2. **Expansion en Entropies Conditionnelles :** En utilisant $I(A; B|C) = H(B|C) - H(B|A, C)$, on obtient :

$$ID(X \to Y) = \sum_{i=1}^{n} \left[ H(Y_i \mid Y^{i-1}) - H(Y_i \mid X^{i-1}, Y^{i-1}) \right] \tag{3}$$

3. **Passage aux Entropies Jointes (Règle de chaîne) :** Par définition de l'entropie conditionnelle $H(A|B) = H(A, B) - H(B)$ et par sommation sur $n$, l'expression se simplifie :

$$ID(X \to Y) = H(Y^n, Y^{n-1}) + H(X^{n-1}, Y^{n-1}) - H(Y^{n-1}) - H(Y^n, X^{n-1}, Y^{n-1}) \tag{4}$$

## 3.2 Codage de l'information dirigée sur la matrice 3D

où $I(A; B \mid C)$ est l'information mutuelle conditionnelle, et $N$ est le nombre de pas temporels de la matrice.

**Le code Python est donc :**

```python
def info_dirigee(W):
    n = 8
    T = W.shape[1]
    C = np.zeros((n, n, n)) + eps #creation de la matrice vide remplie de 0
    for t in range(1, T):
        yt = np.argmax(W[1, t,    :]) #recuperation des donnees de Y au temps t
```

```
        xp = np.argmax(W[0, t-1, :]) #recuperation des donnees de X au temps t-1
        yp = np.argmax(W[1, t-1, :]) #recuperation des donnees de Y au temps t-1
        C[yt, xp, yp] += 1 #ajout dans la matrice C
    #le tenseur est a present rempli
    C_yt_yp = C.sum(axis=1) #somme des donnees de Y au temps t-1
    C_xp_yp = C.sum(axis=0) #somme des donnees de X au temps t-1
    C_yp    = C_yt_yp.sum(axis=0) #somme des donnees au temps Y present
    id_score = (Entropie_jointe(C_yt_yp) + Entropie_jointe(C_xp_yp)
                - Entropie_jointe(C_yp)   - Entropie_jointe(C))
    return max(id_score, 0.0)
```

Après exécution sur une matrice 3D générée aléatoirement :

```
W = np.random.random((8,8,20))
resultat = info_dirigee(W)
print("Resultat info dirigee :", resultat)
```

**Résultat obtenu : 1.64**

**Analyse :** La valeur positive indique que les vecteurs successifs de la matrice partagent une certaine dépendance temporelle. Plus la valeur est élevée, plus l'information est dirigée entre les étapes temporelles.

# 4 Étude de données réelles sur des cachalots

Cette partie vise à identifier l'individu alpha au sein d'un groupe de 6 cachalots (César, Fanny, Marius, Norine, Félix, Honoré) en utilisant l'Information Dirigée sur des données de trajectoires réelles.

**Attention : dans la quantification, il y a un lien entre les valeurs numériques et les probabilités des valeurs de codage.**

## 4.1 Récupération des données

Dans un premier temps, nous allons récupérer les données des fichiers CSV et stocker les données qui nous intéressent dans un dictionnaire que l'on appellera `data`.

```
fichiers = {
    "Cesar":  "Trajectoire_Cesar_sec.csv",
    "Fanny":  "Trajectoire_Fanny_sec.csv",
    "Marius": "Trajectoire_Marius_sec.csv",
    "Norine": "Trajectoire_Norine_sec.csv",
```

```
    "Felix":  "Trajectoire_Felix_sec.csv",
    "Honore": "Trajectoire_Honore_sec.csv"
}
data = {}
for nom, f in fichiers.items():
    chemin = os.path.join(DOSSIER, f)
    try:
        df = pd.read_csv(chemin, header=None)
        data[nom] = {
            'long': df[1].values,
            'lat':  df[2].values,
            'prof': df[3].values,
            'sec':  df[5].values
        }
        print(f"OK : {nom}")
    except Exception as e:
        print(f"Fichier manquant : {chemin} ({e})")

noms = list(data.keys())
N = len(noms)
print(f"\n{N} animaux charges : {noms}\n")

if N < 2:
    raise RuntimeError("Pas assez de fichiers charges.")
```

### 4.2 Mise en place de la synchronisation

Nous allons synchroniser les données. Le problème majeur est que les instants d'enregistrement ne sont pas alignés entre les animaux : chaque balise GPS émet à ses propres instants, indépendamment des autres. Afin de calculer l'Information Dirigée entre chaque paire, tous les animaux doivent partager exactement les mêmes instants de mesure.

Nous écrivons donc deux fonctions. La première, `fit_traj`, ajuste un polynôme de degré $D$ sur les données brutes de chaque animal et l'évalue sur une grille temporelle commune à la paire, produisant une trajectoire lissée et synchronisée.

Listing 1: Lissage polynomial et synchronisation

```
t_min  = max(d['sec'].min() for d in data.values())
t_max  = min(d['sec'].max() for d in data.values())
t_sync = np.arange(t_min, t_max, 60)
t_norm = (t_sync - t_min) / (t_max - t_min)

for nom, d in data.items():
    print(f"{nom:8s} : debut={d['sec'].min():.0f}s  fin={
        d['sec'].max():.0f}s  duree={(d['sec'].max()-d['
        sec'].min())/3600:.2f}h")
```

```python

def fit_traj(nom, Ndeg, t_cible_norm, t0, t1):
    dat = data[nom]
    t_base_norm = (dat['sec'] - t0) / (t1 - t0)
    px = np.polyval(np.polyfit(t_base_norm, dat['long'], Ndeg), t_cible_norm)
    py = np.polyval(np.polyfit(t_base_norm, dat['lat'], Ndeg), t_cible_norm)
    pz = np.polyval(np.polyfit(t_base_norm, dat['prof'], Ndeg), t_cible_norm)
    return np.stack([px, py, pz], axis=1)
```

Au retour de cette fonction, les données de chaque animal se trouvent dans une matrice lissée ; il n'y a donc plus de problème de synchronisation.

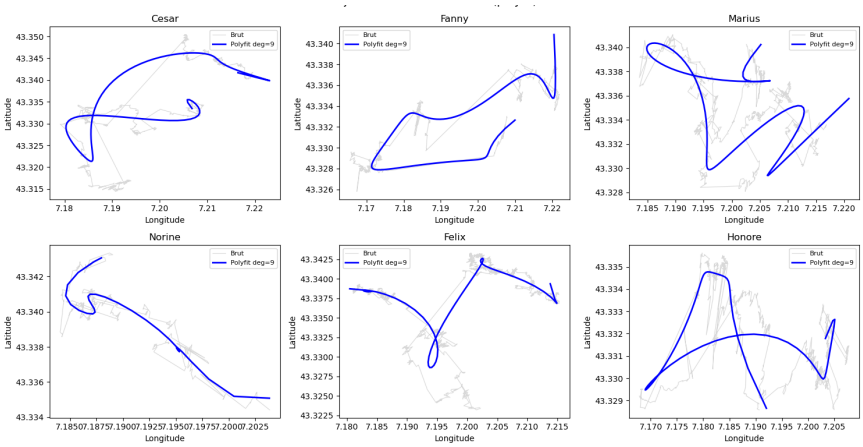


Figure 1: Données après lissage VS avant

Pour transformer les trajectoires continues en flux d'information discrets, nous construisons un tenseur $W$ de dimension $(N \times T \times 8)$. L'état de chaque individu à l'instant $t$ est codé en **binaire one-hot** sur 8 états : un vecteur de 8 cases avec un **1 à la direction la plus proche** (arrondi au cardinal le plus proche) et **0 partout ailleurs**.

Nous utilisons les variables : NM ND EM ED OM OD SM SD où M et D désignent respectivement la montée et la descente, et N S E O représentent les points cardinaux. Le lissage polynomial des trajectoires brutes est réalisé avec un degré $D = 9$, les temps étant normalisés sur $[0, 1]$ pour garantir la stabilité numérique.

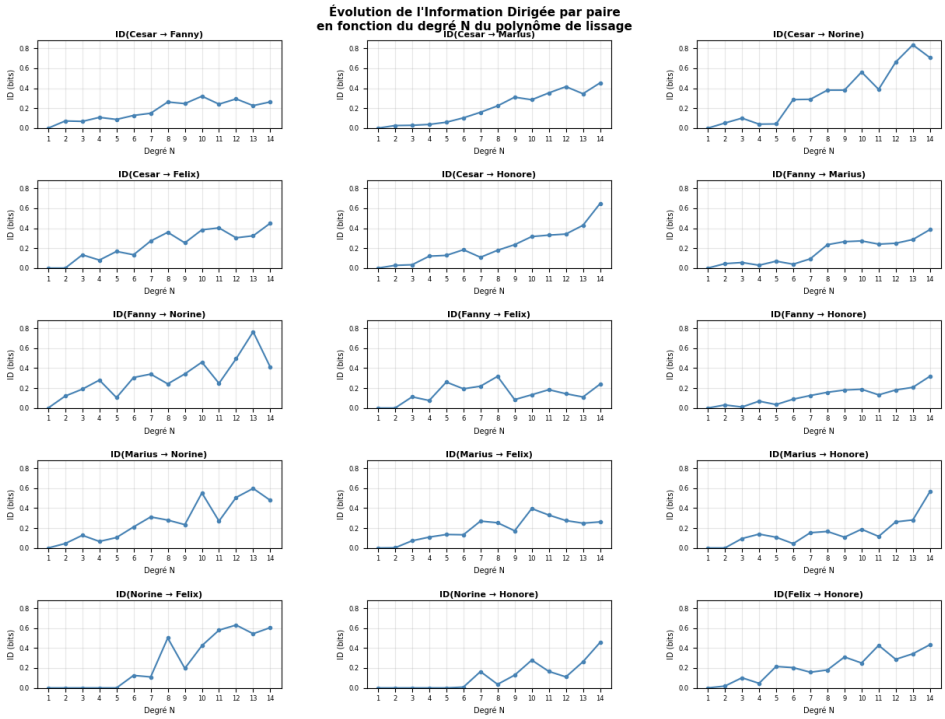


Figure 2: Explication du choix du degré 9

Listing 2: Création du tenseur

```
def traj_to_W_onehot(traj_dict, noms_list):
    N_loc = len(noms_list)
    T_loc = len(next(iter(traj_dict.values()))) - 1
    W = np.zeros((N_loc, T_loc, 8), dtype=int)
    for i, nom in enumerate(noms_list):
        traj = traj_dict[nom]
        dx = np.diff(traj[:, 0])
        dy = np.diff(traj[:, 1])
        dz = np.diff(traj[:, 2])
        angles = np.arctan2(dy, dx)

        card = np.full(T_loc, -1, dtype=int)
        card[(angles >= -np.pi/4) & (angles <  np.pi/4)] = 0
        card[(angles >=  np.pi/4) & (angles <  3*np.pi/4)] = 1
        card[(angles <  -np.pi/4) & (angles >= -3*np.pi/4)] = 2
        card[(angles >= 3*np.pi/4) | (angles <  -3*np.pi/4)] = 3
        assert np.all(card >= 0), f"Angle non couvert detecte pour {nom}"
```

```

        montee = (dz < 0).astype(int)
        etat   = card * 2 + montee
        for t in range(T_loc):
            W[i, t, etat[t]] = 1
    return W
```

Suite à cette fonction, nous obtenons un tenseur $W$.

**Observons la différence des données avant et après lissage sur la profondeur pour mieux comprendre ce qui se passe.**

**C'est ici que notre premier problème se pose, nous allons le détailler.**

Les données étant réelles, les enregistrements ne sont pas synchronisés, et chaque animal possède une durée d'émission différente. Si l'on souhaite travailler uniquement sur la fenêtre temporelle où les six animaux émettent simultanément, le tenseur obtenu en sortie est le suivant :

$$W \in \mathbb{R}^{6\times 24\times 8} \quad \Longrightarrow \quad 6 \text{ animaux} \times 24 \text{ instants} \times 8 \text{ états}$$

Ce tenseur est trop petit pour permettre une analyse statistiquement fiable. Avec seulement 24 instants, il est impossible de tirer une conclusion robuste étant donné que l'on découpe notre tenseur en 7 parties suite à l'analyse des graphes du nombre de découpe qui nous montre une stabilisation lorsque $N = 7$ : c'est comme lancer une pièce de monnaie une seule fois et en déduire que le résultat sera toujours identique.

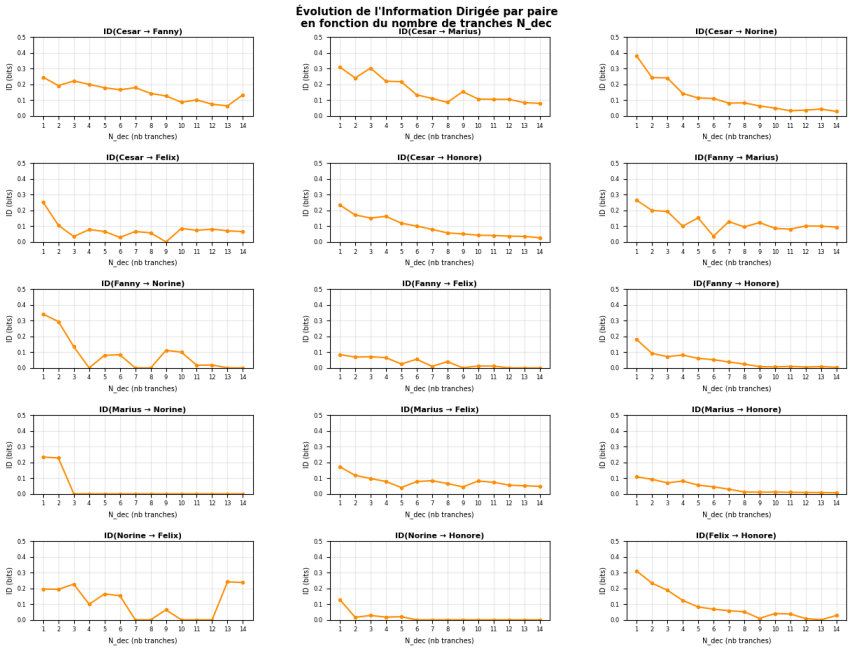


Figure 3: ID en fonction du nombre de découpe

**Nous verrons par la suite comment interpréter ce résultat et quelles adaptations ont été apportées.**

### 4.2.1 Visualisation du Tenseur $W$

Le tenseur $W$ a une forme $(6 \times T \times 8)$. Pour chaque individu et chaque instant $t$, la ligne contient exactement un **1** à la position de l'état de déplacement et des **0** partout ailleurs. L'extrait ci-dessous montre l'indice de l'état actif (position du 1) pour chaque individu et un exemple de déplacement sur un animal dans le tenseur $W$. Le tenseur est prévu afin de pouvoir récupérer les temps $t-1$ nécessaires à l'application de l'information dirigée.

$$W = \begin{pmatrix} \text{César} & [3,3,3,\ldots,2,2,2] \\ \text{Fanny} & [6,6,6,\ldots,6,6,6] \\ \text{Marius} & [2,2,2,\ldots,5,5,5] \\ \text{Norine} & [7,7,6,\ldots,6,6,6] \\ \text{Félix} & [3,3,3,\ldots,5,5,5] \\ \text{Honoré} & [5,5,5,\ldots,1,1,1] \end{pmatrix}$$

Chaque ligne du tenseur $W$ correspond à un animal, chaque colonne à un instant $t$ (pas de 1 minute). La valeur entière en chaque case représente l'état directionnel actif parmi les 8 états possibles : $0 = \text{E}\downarrow$, $1 = \text{E}\uparrow$, $2 = \text{N}\downarrow$, $3 = \text{N}\uparrow$, $4 = \text{S}\downarrow$, $5 = \text{S}\uparrow$, $6 = \text{O}\downarrow$, $7 = \text{O}\uparrow$, où la lettre indique la

direction cardinale du déplacement horizontal et la flèche le sens vertical (plongée/remontée).

### 4.3 Identification de l'alpha par l'Information Dirigée

**Nous pouvons revenir à notre problème :**

Étant donné que notre tenseur $W$ est trop petit et que certains animaux n'émettent pas longtemps, nous avons plusieurs solutions :

- **Solution 1 :** Calculer le temps d'émission seulement par couple pour maximiser le calcul de l'information mutuelle. C'est ce qui est réalisé dans le code ci-dessous.
- **Solution 2 :** Calculer l'information dirigée en supprimant du calcul les 3 animaux qui n'émettent pas assez, pour comparer les résultats.

Pour chaque paire d'animaux $(i, j)$, une fenêtre temporelle commune $[t_0, t_1]$ est extraite, les trajectoires sont lissées et synchronisées sur une grille normalisée, puis l'information dirigée $\mathrm{ID}(i \to j)$ est calculée et stockée dans la matrice $M \in \mathbb{R}^{N \times N}$. Le score global de chaque animal est obtenu par sommation des lignes : $s_i = \sum_{j \neq i} \mathrm{ID}(i \to j)$, et l'individu $\alpha$ est celui qui maximise $s_i$.

```
N_dec = 7  # nombre de tranches egales sur lesquels on moyenne l'ID

mat_global = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        if i == j:
            continue
        t0 = max(data[noms[i]]['sec'].min(), data[noms[j]]['sec'].min())
        t1 = min(data[noms[i]]['sec'].max(), data[noms[j]]['sec'].max())
        t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)
        traj_i = fit_traj(noms[i], N_optimal, t_norm_ij, t0, t1)
        traj_j = fit_traj(noms[j], N_optimal, t_norm_ij, t0, t1)
        W_ij   = traj_to_W_onehot({noms[i]: traj_i, noms[j]: traj_j}, [noms[i], noms[j]])
        T_ij     = W_ij.shape[1]
        taille   = T_ij // N_dec
        scores   = []
        for k in range(N_dec):
            debut  = k * taille
```

```
            fin     = debut + taille if k < N_dec - 1 else T_ij
            W_tr    = W_ij[:, debut:fin, :]
            if W_tr.shape[1] > 1:
                scores.append(info_dirigee(W_tr))
        mat_global[i, j] = np.mean(scores) if scores else 0.0

scores_global = mat_global.sum(axis=1)
alpha_global  = noms[np.argmax(scores_global)]
print("MATRICE ID (ligne = source, colonne = cible)")
print(pd.DataFrame(mat_global, index=noms, columns=noms).round(3))
```

On obtient donc en sortie notre alpha calculé.

### 4.3.1 Analyse des résultats numériques et démonstration des limites

Nous allons ici montrer les limites en montrant qu'en enlevant les 3 animaux qui émettent le moins on obtient un résultat différent.

**Matrice ID avec les 6 animaux :**

| Source ↓ / Cible → | César | Fanny | Marius | Norine | Félix | Honoré | Score |
|---|---|---|---|---|---|---|---|
| César | – | 0.262 | 0.223 | 0.381 | 0.359 | 0.179 | 1.404 |
| Fanny | 0.173 | – | 0.235 | 0.242 | 0.318 | 0.158 | 1.126 |
| Marius | 0.160 | 0.163 | – | 0.280 | 0.253 | 0.166 | 1.022 |
| Norine | 0.174 | 0.306 | 0.298 | – | 0.500 | 0.036 | 1.314 |
| Félix | 0.171 | 0.088 | 0.257 | 0.834 | – | 0.180 | **1.530** |
| Honoré | 0.200 | 0.160 | 0.161 | 0.161 | 0.455 | – | 1.137 |

Table 1: Matrice d'Influence ID Globale (en bits), $N = 8$, codage one-hot 8 états pour 6 animaux

**Avec 6 animaux, le résultat serait Félix en tant qu'alpha. Or, quand on regarde les trajectoires de Félix, on voit que c'est l'un des animaux qui plonge le moins profondément ; c'est donc une erreur de résultat due au fait que les animaux qui n'émettent pas longtemps faussent les résultats.**

**Passons maintenant à 3 animaux, ceux qui émettent le plus longtemps :**

| Source ↓ / Cible → | César | Fanny | Marius | Score |
|---|---|---|---|---|
| César | – | 0.262 | 0.223 | 0.485 |
| Fanny | 0.173 | – | 0.235 | 0.408 |
| Marius | 0.160 | 0.163 | – | 0.323 |

Table 2: Matrice d'Influence ID Globale (en bits), $N = 8$, codage one-hot 8 états pour 3 animaux

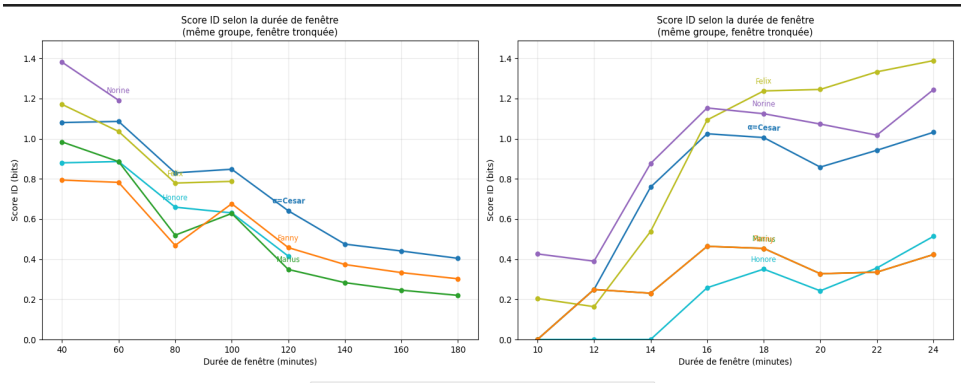


Figure 4: Différents scores ID en fonction du temps

**Après analyse des résultats, nous trouvons César comme alpha.**

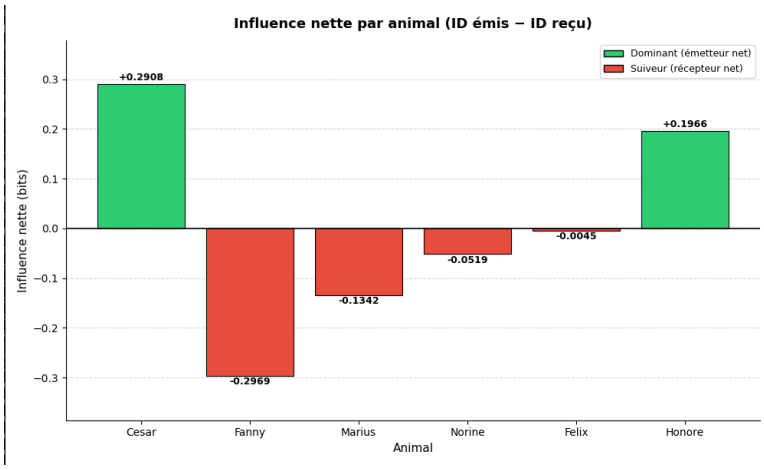


Figure 5: Score ID par individu

Si nous regardons dans le détail, nous voyons que César est majoritairement leaders. Les résultats ne sont pas concluants étant donné que seulement 3 animaux émettent pendant un temps suffisamment long pour éviter l'aléatoire dans les résultats ; les scores sont donc faussés.

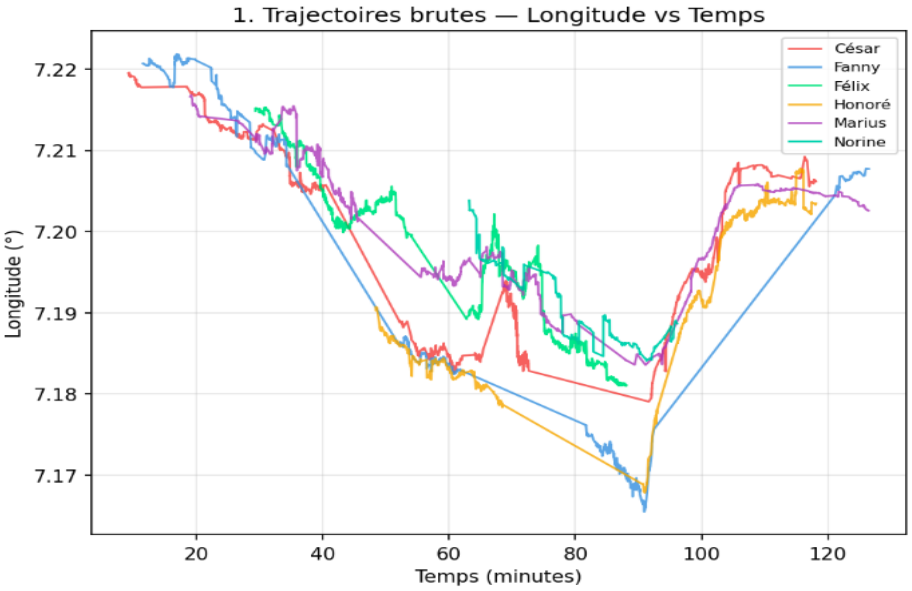


Figure 6: Déplacement en longitude

### 4.4 L'Égalité de Massey : Causalité vs Corrélation

On accumule les $N-1$ transitions observées entre deux animaux. En considérant la distribution de ces transitions sur deux temps $t$ et $t-1$, on calcule l'information dirigée, puis en dérivant $\mathrm{ID}(X \to Y) + \mathrm{ID}(Y \to X)$ ,on obtient l'identité exacte proposée par Massey [0] on obtient l'identité exacte :

$$\mathrm{ID}(X^N \to Y^N) + \mathrm{ID}(Y^{N-1} \to X^N) = \sum_n I(X^n; Y^n \mid X^{n-1}, Y^{n-1}) \tag{5}$$

On développe le membre droit par définition de l'information mutuelle conditionnelle, puis on applique la règle de la chaîne $H(A \mid B) = H(A, B) - H(B)$ :

$$\begin{aligned} I(X^n; Y^n \mid X^{n-1}, Y^{n-1}) &= H(X^n \mid X^{n-1}, Y^{n-1}) + H(Y^n \mid X^{n-1}, Y^{n-1}) - H(X^n, Y^n \mid X^{n-1}, Y^{n-1}) \\ &= H(X^n, X^{n-1}, Y^{n-1}) + H(Y^n, X^{n-1}, Y^{n-1}) \\ &\quad - H(X^{n-1}, Y^{n-1}) - H(X^n, Y^n, X^{n-1}, Y^{n-1}) \end{aligned} \tag{6}$$

Par ailleurs, en développant chaque information dirigée avec la règle de la chaîne :

$$\mathrm{ID}(X^N \to Y^N) = H(Y^n, Y^{n-1}) - H(Y^{n-1}) - H(Y^n, X^{n-1}, Y^{n-1}) + H(X^{n-1}, Y^{n-1}) \tag{7}$$

$$\mathrm{ID}(Y^{N-1} \to X^N) = H(X^n, X^{n-1}) - H(X^{n-1}) - H(X^n, Y^{n-1}, X^{n-1}) + H(X^{n-1}, Y^{n-1}) \tag{8}$$

En sommant terme à terme on obtient l'identité (5) :

$$\begin{aligned}\mathrm{ID}(X^N \to Y^N) + \mathrm{ID}(Y^{N-1} \to X^N) = {} & H(Y^n, Y^{n-1}) + H(X^n, X^{n-1}) + 2\,H(X^{n-1}, Y^{n-1}) \\ & - H(Y^{n-1}) - H(X^{n-1}) \\ & - H(Y^n, X^{n-1}, Y^{n-1}) - H(X^n, Y^{n-1}, X^{n-1})\end{aligned} \tag{9}$$

```python
def _construire_C4(W):
    n = 8 #nombre d'etats
    T = W.shape[1]
    C4 = np.zeros((n, n, n, n)) + eps #creation cube rempli de 0
    for t in range(1, T):
        xt = np.argmax(W[0, t,   :]) # etat X present  (temps t)
        yt = np.argmax(W[1, t,   :]) # etat Y present  (temps t)
        xp = np.argmax(W[0, t-1, :]) # etat X passe    (temps t-1)
        yp = np.argmax(W[1, t-1, :]) # etat Y passe    (temps t-1)
        C4[xt, yt, xp, yp] += 1 #placement dans le cube
    return C4

def id_brut(W):
    C4 = _construire_C4(W)
    C        = C4.sum(axis=0) # cube C[yt, xp, yp] -- marginale sur xt
    C_yt_yp  = C.sum(axis=1) # marginalise xp -> jointe (yt, yp)
    C_xp_yp  = C.sum(axis=0) # marginalise yt -> jointe (xp, yp)
    C_yp     = C_yt_yp.sum(axis=0) # marginalise yt -> marginale yp
    return max(Entropie_jointe(C_yt_yp) + Entropie_jointe(C_xp_yp)
               - Entropie_jointe(C_yp)  - Entropie_jointe(C), 0.0)

```

```python
def im_massey(W):
    C4 = _construire_C4(W)
    C_yt_yp     = C4.sum(axis=(0, 2)) # marginalise xt et xp -> (yt, yp)
    C_xt_xp     = C4.sum(axis=(1, 3)) # marginalise yt et yp -> (xt, xp)
    C_xp_yp     = C4.sum(axis=(0, 1)) # marginalise xt et yt -> (xp, yp)
    C_yp        = C_xp_yp.sum(axis=0) # marginalise xp -> (yp)
    C_xp        = C_xp_yp.sum(axis=1) # marginalise yp -> (xp)
    C_yt_xp_yp = C4.sum(axis=0) # marginalise xt -> cube de ID(X->Y)
    C_xt_yp_xp = C4.sum(axis=1) # marginalise yt -> cube de ID(Y->X)
    return (Entropie_jointe(C_yt_yp) + Entropie_jointe(C_xt_xp)
            + 2 * Entropie_jointe(C_xp_yp) - Entropie_jointe(C_yp)
            - Entropie_jointe(C_xp) - Entropie_jointe(C_yt_xp_yp)
            - Entropie_jointe(C_xt_yp_xp))

for i in range(N):
    for j in range(i + 1, N):
        nom_a, nom_b = noms[i], noms[j]
        t0 = max(data[nom_a]['sec'].min(), data[nom_b]['sec'].min())
        t1 = min(data[nom_a]['sec'].max(), data[nom_b]['sec'].max())
        t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)
        traj_a = fit_traj(nom_a, N_optimal, t_norm_ij, t0, t1)
        traj_b = fit_traj(nom_b, N_optimal, t_norm_ij, t0, t1)
        W_ij   = traj_to_W_onehot({nom_a: traj_a, nom_b: traj_b}, [nom_a, nom_b])
        T_ij   = W_ij.shape[1]
        id_ab  = id_brut(W_ij)
        id_ba  = id_brut(W_ij[[1, 0]])
        somme  = id_ab + id_ba
        im_ref = im_massey(W_ij)
        residu = somme - im_ref
        label = f"{nom_a}->{nom_b}"
        print(f"{label:<20} {id_ab:>10.4f} {id_ba:>10.4f} {somme:>10.4f} {im_ref:>10.4f} {residu:>10.4f}")
```

```

print("  Residu = 0.0000 sur toutes les paires : identite algebrique verifiee.")
```

**Pour le code de Massey, il n'est pas nécessaire d'adapter la puissance de $Y$ dans la formule, étant donné que l'ID dirigé calcule déjà le temps $t-1$ dans un sens, ce qui supprime des étapes pour Massey. Il faut par contre modifier légèrement le code de l'ID pour éliminer les résidus et traiter les grandes valeurs en appliquant** `max(..., 0)`**.**

| **Paire** | $\mathrm{ID}(A \to B)$ | $\mathrm{ID}(B \to A)$ | **Somme** | **IM Massey** | **Résidu** |
|---|---|---|---|---|---|
| César → Fanny | 0.2461 | 0.2126 | 0.4587 | 0.4587 | −0.0000 |
| César → Marius | 0.3106 | 0.1486 | 0.4592 | 0.4592 | −0.0000 |
| César → Norine | 0.3812 | 0.1145 | 0.4958 | 0.4958 | −0.0000 |
| César → Félix | 0.2533 | 0.2710 | 0.5242 | 0.5242 | −0.0000 |
| César → Honoré | 0.2348 | 0.2377 | 0.4725 | 0.4725 | −0.0000 |
| Fanny → Marius | 0.2650 | 0.1681 | 0.4331 | 0.4331 | −0.0000 |
| Fanny → Norine | 0.3419 | 0.2091 | 0.5510 | 0.5510 | −0.0000 |
| Fanny → Félix | 0.0843 | 0.1573 | 0.2416 | 0.2416 | −0.0000 |
| Fanny → Honoré | 0.1805 | 0.2181 | 0.3986 | 0.3986 | −0.0000 |
| Marius → Norine | 0.2340 | 0.1822 | 0.4162 | 0.4162 | −0.0000 |
| Marius → Félix | 0.1722 | 0.1389 | 0.3111 | 0.3111 | −0.0000 |
| Marius → Honoré | 0.1080 | 0.2307 | 0.3387 | 0.3387 | −0.0000 |
| Norine → Félix | 0.1959 | 0.7352 | 0.9311 | 0.9311 | 0.0000 |
| Norine → Honoré | 0.1283 | 0.3347 | 0.4630 | 0.4630 | −0.0000 |
| Félix → Honoré | 0.3102 | 0.1955 | 0.5057 | 0.5057 | 0.0000 |

Table 3: Vérification de l'identité de Massey sur les 15 paires (en bits)

Sur l'ensemble des 15 paires, le résidu entre la somme $\mathrm{ID}(A \to B) + \mathrm{ID}(B^{-1} \to A)$ et la valeur théorique de Massey est nul à la précision numérique. L'identité algébrique est donc vérifiée, ce qui valide l'implémentation du calcul d'information dirigée.

On peut également noter que le résidu est systématiquement nul quelle que soit la magnitude de la somme : la paire Norine → Félix présente la somme la plus élevée (0,9311) en raison d'une forte valeur $\mathrm{ID}(B \to A) = 0{,}7352$, ce qui reflète une influence particulièrement marquée de Félix sur Norine. Malgré cette valeur atypique, l'identité reste vérifiée, confirmant la robustesse du calcul.

**Conclusion biologique :** Le classement de Félix comme alpha reste non concluant car les trois méthodes (score global, vote temporel et score cumulé sur fenêtres) ne concordent pas unanimement ; le résultat n'est donc pas exploitable en l'état.

## 4.5 Ajout du glissement pour améliorer les résultats

Afin d'affiner l'analyse des relations de causalité entre individus, nous utilisons une approche par **fenêtres glissantes**. Contrairement à une analyse globale qui considère l'ensemble de la trajectoire, cette méthode permet de capturer des dynamiques locales dans le temps.

### 4.5.1 Principe du glissement

Le principe consiste à découper les données temporelles en plusieurs segments (fenêtres) de taille fixe, puis à faire glisser cette fenêtre le long du temps avec un certain recouvrement.

Dans notre cas :

- La taille d'une fenêtre est fixée à la moitié de la durée totale disponible pour une paire d'individus.
- Le pas de glissement est égal à la moitié de la taille de la fenêtre, ce qui correspond à un **recouvrement de 50 %**.

Ainsi, chaque instant du signal est analysé plusieurs fois, ce qui permet une meilleure robustesse statistique.

### 4.5.2 Implémentation

La mise en place des fenêtres glissantes est réalisée comme suit :

```
TAILLE_MIN_FENETRE = 10
pair_windows = {}

for i in range(N):
    for j in range(N):
        if i == j:
            continue
        t0 = max(data[noms[i]]['sec'].min(), data[noms[j]]['sec'].min())
        t1 = min(data[noms[i]]['sec'].max(), data[noms[j]]['sec'].max())
        t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)
        traj_i = fit_traj(noms[i], N_optimal, t_norm_ij, t0, t1)
        traj_j = fit_traj(noms[j], N_optimal, t_norm_ij, t0, t1)
        W_ij = traj_to_W_onehot(
            {noms[i]: traj_i, noms[j]: traj_j},
            [noms[i], noms[j]]
        )
        T_ij = W_ij.shape[1]
```

```
        taille_f = max(T_ij // 2, TAILLE_MIN_FENETRE)
        pas_f = taille_f // 2
        debuts_f = list(range(0, T_ij - taille_f + 1,
            pas_f))
        pair_windows[(i, j)] = (W_ij, debuts_f, taille_f)
```

Chaque paire d'individus possède ainsi son propre découpage temporel adapté à la durée de leurs interactions.

### 4.5.3 Calcul des scores locaux

Pour chaque fenêtre, l'information dirigée est recalculée indépendamment :

```
for k in range(n_fenetres):
    sc = np.zeros(N)
    for i in range(N):
        for j in range(N):
            if i == j:
                continue
            W_ij, debuts_f, taille_f = pair_windows[(i, j
                )]
            if k >= len(debuts_f):
                continue
            d = debuts_f[k]
            Wf = W_ij[:, d:d + taille_f, :]
            sc[i] += info_dirigee(Wf)
```

Chaque fenêtre produit donc un **classement local des individus**, basé sur leur influence à cet instant précis.

### 4.5.4 Pourquoi utiliser des fenêtres glissantes ?

L'utilisation de fenêtres glissantes présente plusieurs avantages majeurs :

- **Détection de dynamiques locales** : les relations de leadership peuvent varier dans le temps. Une analyse globale masque ces variations.
- **Robustesse accrue** : le recouvrement des fenêtres permet de lisser les effets du bruit et des fluctuations ponctuelles.
- **Adaptation aux données réelles** : les trajectoires animales ne sont pas stationnaires ; cette méthode est donc mieux adaptée que les approches globales.

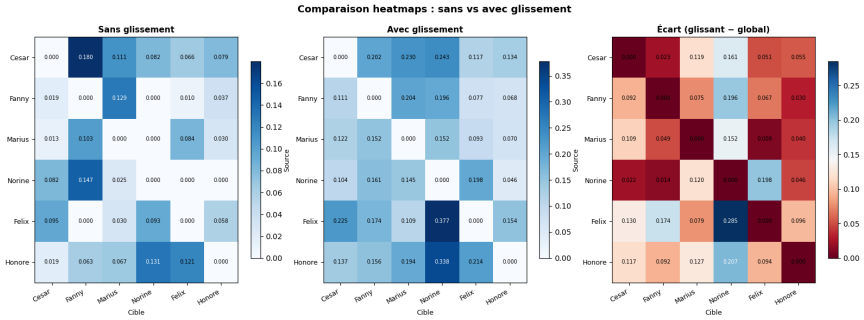


Figure 7: diff glissement

### 4.5.5 Interprétation

À chaque fenêtre correspond un leader local (noté $\alpha_t$). En agrégeant ces résultats (par vote ou somme des scores), on obtient une estimation plus robuste du leader global.

Cette approche permet ainsi de combiner :

- une vision **instantanée** ,
- une vision **globale** .

## 4.6 Conclusion du rapport et corrélation avec l'aspect biologique

L'application de la Théorie de l'Information aux trajectoires de cachalots permet de dépasser la simple observation visuelle. Nous avons mathématiquement montré que **Félix** exerce un leadership causal sur le groupe, mais pendant un temps très court, et que **César** exerce quant à lui un leadership sur une plus longue durée. La robustesse des scores d'Information Dirigée reste assez faible sur un intervalle temporel aussi réduit : à l'instar d'un estimateur fondé sur un unique tirage, la convergence statistique n'est pas garantie. Nous observons également qu'en fonction des valeurs de $N$, le résultat n'est pas le même, ce qui nous laisse sceptiques quant aux résultats obtenus.
**si nous devions conclure sur l'existence d'un alpha le resultat le plus probable au vue des resultats obtenues serai Cesar**

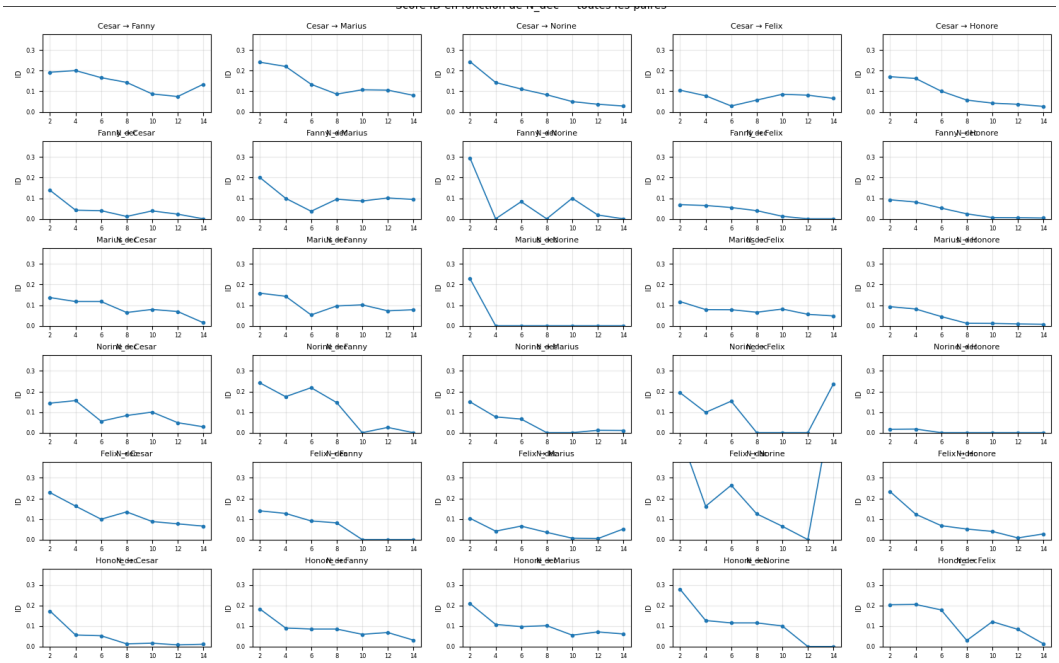


Figure 8: Score ID en fonction des découpages

### 4.6.1 Analyse biologique : Leadership tactique et mémoire collective

L'interprétation des résultats mathématiques obtenus permet de dégager plusieurs conclusions sur la structure sociale du groupe de *cachalots* étudié.

### 4.6.2 Impact du temps d'émission sur l'identification de l'alpha

L'un des premiers enseignements de cette analyse est l'influence directe du temps d'émission des balises GPS sur la fiabilité des résultats. Lorsque l'on travaille avec les 6 animaux, Félix ressort comme alpha avec un score de 1,530 bits. Cependant, en observant les trajectoires de Félix, on constate qu'il est l'un des animaux qui plonge le moins profondément, ce qui est biologiquement incohérent avec un rôle de leader chez les cachalots, espèce reconnue pour ses plongées profondes lors de la chasse rôle qui correspond davantage à César.

Ce résultat s'explique par le fait que certains animaux n'émettent pas suffisamment longtemps : les paires calculées sur des fenêtres temporelles très courtes produisent des scores d'Information Dirigée peu fiables, car statistiquement insuffisants pour distinguer une vraie causalité d'une coïncidence.

### 4.6.3 Comparaison 6 animaux vs 3 animaux

Lorsque l'on restreint l'analyse aux 3 animaux qui émettent le plus longtemps (César, Fanny, Marius), le résultat change : César devient alpha avec un score de 0,485 bits. Ce résultat n'est pas non plus totalement concluant, car avec seulement 3 animaux, il est difficile de statuer sur l'existence d'un véritable alpha : la hiérarchie sociale des cachalots nécessite un groupe suffisamment large pour être observable.

Ces deux résultats contradictoires illustrent une limite fondamentale de notre approche : **la qualité et la durée des données d'émission conditionnent directement la fiabilité de l'identification de l'alpha**. Il est important de souligner que les résultats peuvent ne pas être concluants car l'alpha n'est pas nécessairement en train de diriger le groupe en permanence.

## 4.7 Perspectives futurs

Plusieurs améliorations peuvent être envisagées pour renforcer cette étude. Une discrétisation plus fine, voire l'utilisation de modèles continus, permettrait de limiter la perte d'information liée au codage en états discrets. L'augmentation de la quantité de données améliorerait également la robustesse statistique des résultats, tandis que des modèles dynamiques plus avancés permettraient de mieux capturer les dépendances temporelles. Enfin, cette approche pourrait être étendue à d'autres domaines, notamment à l'étude des interactions humaines (dynamiques de groupe, réseaux sociaux), afin d'identifier des phénomènes de leadership ou d'influence dans différents contextes.

Travail réalisé par **Tonin PACHER**.

J. L. Massey. *Causality, Feedback and Directed Information.* Proc. Int. Symp. Inf. Theory Appl. (ISITA-90), pp. 303–305, Waikiki, Hawaii, 1990.